

Tema 7.

Extracción automática de terminología

Candidatos terminológicos, n-gramas, weidness, KWIC y miniglosarios con Python

Material teórico | Programación para PLN | Universidad Pontificia Comillas

Dr.^a Blanca Hernández Pardo

Índice de contenidos

Título narrativo.....	1
Contexto profesional.....	1
Encargo: apoyar la creación de glosarios en LexiData Solutions.....	1
El caso de trabajo.....	1
Qué vas a aprender en esta unidad.....	2
Mapa conceptual de la unidad.....	2
1. Marco teórico.....	3
1.1. Qué es un término.....	3
1.2. Terminología, traducción y gestión del conocimiento.....	3
1.3. Extracción automática de términos: ATE y ATR.....	4
1.4. Termhood y unithood.....	4
1.5. Corpus especializado y corpus general.....	5
1.6. Frecuencia absoluta y frecuencia relativa.....	5
1.7. Weirdness: rareza relativa y especificidad de dominio.....	6
1.8. N-gramas y términos multipalabra.....	6
1.9. Filtrado: frecuencia mínima, stopwords y listas negras.....	7
1.10. Métodos transparentes frente a métodos neuronales.....	7
1.11. KWIC y validación humana.....	7
1.12. Evaluación y límites de la extracción automática.....	8
2. Extracción automática de terminología con Python.....	8
2.1. Preparar el entorno.....	8
2.2. Representar los corpus.....	9
2.3. Tokenización simple.....	9
2.4. Stopwords y filtrado funcional.....	10
2.5. Preprocesar un corpus completo.....	10
2.6. Extraer n-gramas.....	10
2.7. Contar n-gramas con Counter.....	11
2.8. Convertir n-gramas a texto.....	11
2.9. Filtrar por frecuencia mínima.....	11
2.10. Calcular tamaños de corpus y frecuencia relativa.....	12

2.11. Calcular weirdness.....	13
2.12. Aplicar weirdness a unigramas, bigramas y trigramas.....	14
2.13. Unificar candidatos de distinta longitud.....	14
2.14. Refinar con una lista negra.....	14
2.15. Construir KWIC para candidatos.....	15
2.16. Crear un miniglosario automático.....	15
2.17. Imprimir el glosario de forma legible.....	16
2.18. Conservar stopwords dentro de términos.....	16
2.19. Cómo leer los resultados sin adelantar la práctica.....	17
Errores frecuentes de aprendizaje.....	17
Síntesis final.....	17
Cierre narrativo.....	18
Referencias.....	18

Título narrativo

Detectar terminología especializada: de leer documentos a construir candidatos de glosario con Python.

Contexto profesional

Encargo: apoyar la creación de glosarios en LexiData Solutions

Imagina que formas parte de **LexiData Solutions**, una empresa especializada en traducción, localización, documentación técnica y gestión terminológica. Sus clientes trabajan en ámbitos institucionales, científicos, jurídicos, sanitarios y tecnológicos. En todos esos sectores aparece el mismo problema: los equipos de traducción y revisión necesitan glosarios fiables, pero los documentos crecen más rápido que la capacidad humana para leerlos desde cero.

Hasta ahora, la empresa elaboraba muchos glosarios de forma manual. Una terminóloga o un traductor especializado leía documentos, subrayaba unidades importantes, buscaba ejemplos, comprobaba si esas unidades designaban conceptos del dominio y decidía cuáles entraban en la base terminológica. Ese procedimiento sigue siendo necesario, porque la validación terminológica final requiere criterio experto. Pero ya no basta como primer filtro: cada semana llegan manuales, informes, páginas web, memorias institucionales, normativas y documentación multilingüe.

La dirección de LexiData decide crear un prototipo de apoyo. No quiere una herramienta opaca que sustituya al equipo humano. Quiere una herramienta interpretable que genere una lista priorizada de **candidatos a término**. El objetivo profesional es ahorrar tiempo en la fase inicial: localizar palabras y expresiones que parecen propias de un dominio especializado para que una persona experta pueda revisarlas.

El caso de trabajo

El equipo define una simulación de trabajo muy concreta: comparar un corpus especializado con un corpus general. Si una unidad aparece con frecuencia relativa alta en el corpus especializado y aparece poco o nada en el corpus general, puede ser un buen candidato terminológico. Esta idea no garantiza que la unidad sea un término, pero ayuda a priorizar.

El prototipo se basa en métodos transparentes:

1. Preparar dos corpus: uno especializado y otro general.
2. Limpiar y tokenizar los textos.
3. Extraer unigramas, bigramas y trigramas.
4. Filtrar candidatos por frecuencia y stopwords.
5. Calcular frecuencia relativa.
6. Rankear candidatos mediante una medida contrastiva sencilla, weirdness.
7. Crear un miniglosario con ejemplos KWIC para facilitar la validación humana.

La pregunta profesional no es «¿puede Python saber qué términos son correctos?». La pregunta es más útil: «¿puede Python ordenar indicios para que una especialista revise primero los candidatos más prometedores?».

En esta unidad aprenderás a construir esa primera capa de apoyo terminológico. El resultado no será un glosario final, sino una lista razonada de candidatos. La diferencia importa: un candidato puede ser interesante, ruidoso, incompleto o estar deformado por el preprocesamiento. La validación lingüística y terminológica sigue siendo el centro del trabajo.

Qué vas a aprender en esta unidad

Al finalizar esta unidad, deberías ser capaz de:

- explicar qué es un término y por qué no toda palabra frecuente es terminología;
- distinguir entre término, candidato a término, palabra clave y expresión recurrente;
- comprender la diferencia entre frecuencia absoluta y frecuencia relativa;
- explicar por qué se comparan corpus especializados y corpus generales;
- entender qué es un n-grama y por qué muchos términos son multipalabra;
- relacionar termhood y unithood con la extracción automática de terminología;
- implementar un tokenizador simple y transparente con expresiones regulares;
- aplicar stopwords y comprender sus ventajas y riesgos;
- extraer unigramas, bigramas y trigramas con Python;
- contar frecuencias con Counter;
- filtrar candidatos por frecuencia mínima;
- calcular weirdness como medida contrastiva;
- unificar candidatos de distinta longitud en una tabla común;
- depurar candidatos mediante listas negras o heurísticas lingüísticas;
- generar ejemplos KWIC para validar candidatos en contexto;
- explicar por qué un miniglosario automático necesita revisión humana.

Mapa conceptual de la unidad

Bloque	Concepto lingüístico	Operación en Python	Resultado
Corpus especializado	Lenguaje propio de un dominio	Lista de textos o columna textual	Textos del ámbito que interesa
Corpus general	Lenguaje de referencia	Lista de textos comparables	Fondo para contrastar
Tokenización	Unidad mínima de análisis	<code>simple_tokenize()</code>	Lista de tokens
Stopwords	Palabras frecuentes poco informativas	<code>remove_stopwords()</code>	Tokens más orientados a contenido
N-gramas	Secuencias de palabras	<code>get_ngrams(tokens, n)</code>	Unigramas, bigramas y trigramas
Frecuencia absoluta	Apariciones directas	Counter	Conteos por candidato
Frecuencia relativa	Frecuencia normalizada por tamaño	freq / N	Comparación más justa entre corpus
Weirdness	Rareza relativa o especificidad de dominio	Cociente entre frecuencias relativas	Ranking contrastivo
Heurísticas	Reglas interpretables de depuración	Umbral, blacklist, bordes	Candidatos menos ruidosos
KWIC	Contexto de uso	Ventana izquierda/derecha	Ejemplos para validar

1. Marco teórico

1.1. Qué es un término

En terminología, un término no es simplemente una palabra difícil. Un término es una unidad lingüística que designa un concepto dentro de un dominio especializado. Puede ser una sola palabra, como tokenización, o una expresión multipalabra, como modelo de lenguaje, etiquetado morfosintáctico o frecuencia relativa.

Cabré (1999) entiende la terminología como un campo que conecta lenguaje, conocimiento especializado, comunicación profesional y documentación. Desde esta perspectiva, los términos no existen aislados: funcionan dentro de situaciones comunicativas concretas, comunidades expertas, prácticas de traducción, clasificación conceptual y circulación de conocimiento. Cabré (2003) insiste además en que una teoría de la terminología debe explicar no solo la dimensión conceptual, sino también la lingüística y la comunicativa.

Esta idea es esencial para PLN. Un sistema automático puede contar formas y detectar patrones, pero no comprende por sí solo el concepto especializado que una unidad designa. Por eso hablamos de **candidatos a término**. La extracción automática propone unidades que parecen terminológicas; la validación humana decide si realmente lo son.

Unidad	Puede aparecer en un corpus especializado	¿Es necesariamente término?
lenguaje	Sí	No siempre: puede ser palabra general
modelo de lenguaje	Sí	Muy probablemente en PLN
sistema	Sí	Depende del contexto
procesamiento del lenguaje natural	Sí	Sí, si designa el campo o técnica
paso fundamental	Sí	Normalmente no: expresión descriptiva

La extracción terminológica automática trabaja justo en esta zona intermedia: transforma un corpus en una lista priorizada de unidades que merecen revisión.

1.2. Terminología, traducción y gestión del conocimiento

La terminología tiene un papel central en traducción especializada, localización, documentación técnica, normalización y gestión del conocimiento. Un glosario no es solo una lista de palabras: es una herramienta para mantener coherencia conceptual, evitar ambigüedades, facilitar equivalencias multilingües y reducir costes de revisión.

En un proyecto real, un término puede necesitar:

- forma principal;
- variantes;
- categoría gramatical;
- definición;
- dominio;
- equivalencias en otras lenguas;
- ejemplos de uso;
- fuente documental;
- nota de uso;

- estado de validación.

El prototipo de esta unidad no crea toda esa ficha terminológica. Produce una primera macroestructura: una lista de candidatos que pueden convertirse en entradas de glosario. Esta distinción está también en herramientas recientes como Termout, que plantea la producción semiautomática de bases terminológicas a partir de procesamiento de corpus, extracción de terminología y posterior población de datos gramaticales y semánticos (Nazar & Acosta, 2023).

Para una estudiante de lingüística, esta unidad muestra cómo el PLN puede integrarse en un flujo profesional real: no sustituye la terminología, pero acelera la fase de localización y priorización.

1.3. Extracción automática de términos: ATE y ATR

La literatura utiliza expresiones como Automatic Term Extraction (ATE), Automatic Term Recognition (ATR) o automatic terminology extraction. En términos generales, se trata de localizar y rankear unidades lingüísticas que son relevantes para describir un dominio especializado. Kageura y Umino (1996) ya planteaban que los métodos de reconocimiento automático de términos deben apoyarse en una comprensión adecuada de la naturaleza de los términos. No basta con contar palabras.

Tran et al. (2023) definen la extracción automática de términos como una tarea de PLN que reduce el esfuerzo de identificar manualmente términos en corpus de dominio, generando listas de candidatos. También señalan que los términos extraídos son útiles para tareas terminográficas y para aplicaciones posteriores como recuperación de información, traducción automática, detección de temas o análisis de opinión.

La unidad se sitúa dentro de los métodos no supervisados y transparentes. No entrena un clasificador ni usa modelos neuronales. El procedimiento se basa en frecuencia, comparación de corpus, n-gramas y reglas explícitas. Esto es especialmente adecuado para alumnado de Humanidades porque cada decisión puede discutirse.

1.4. Termhood y unithood

Dos conceptos clásicos en extracción terminológica son **termhood** y **unithood**. Kageura y Umino (1996) los presentan como dimensiones centrales para reconocer términos.

Termhood se refiere al grado en que una unidad es propia o relevante para un dominio especializado. Por ejemplo, tokenización puede tener alta termhood en un corpus de PLN porque aparece más en ese dominio que en textos generales.

Unithood se refiere al grado en que una secuencia de palabras funciona como una unidad estable. Por ejemplo, modelo de lenguaje es más que la suma casual de modelo y lenguaje: es una expresión estable que designa un concepto técnico.

Concepto	Pregunta	Ejemplo
Termhood	¿Esta unidad es propia del dominio?	tokenización frente a reunión
Unithood	¿Esta secuencia funciona como unidad?	modelo de lenguaje frente a lenguaje necesita
Candidato	¿Conviene revisarla como posible término?	traducción automática neuronal

El cuaderno de PLN7 trabaja de forma simplificada con ambos conceptos. La comparación entre corpus aproxima termhood. La extracción de bigramas y trigramas aproxima unithood, aunque sin análisis sintáctico profundo.

1.5. Corpus especializado y corpus general

La comparación entre un corpus especializado y un corpus general es una estrategia clásica en extracción de términos. La intuición es sencilla: una unidad terminológica suele aparecer con más densidad en textos del dominio que en textos generales. Esa diferencia permite calcular medidas de especificidad.

Sin embargo, la comparación solo tiene sentido si los corpus están bien pensados. El corpus especializado debe representar el dominio que se quiere estudiar. El corpus general debe funcionar como fondo de contraste. Si el corpus general es demasiado pequeño, demasiado distinto o temáticamente sesgado, la puntuación puede inflarse o distorsionarse.

Verberne et al. (2016) muestran que el tamaño de la colección y la importancia de los términos multipalabra afectan de forma clara al rendimiento de métodos de puntuación terminológica. Su estudio también recuerda que los métodos de extracción no se comportan igual en todos los tipos de colección. Esta advertencia encaja con la unidad: en corpus pequeños, los rankings son didácticos, pero pueden ser inestables.

Por eso, cuando se escribe un informe terminológico, conviene documentar:

- qué textos forman el corpus especializado;
- qué textos forman el corpus general;
- cuántos documentos hay en cada corpus;
- cuántos tokens tiene cada corpus;
- de qué fechas, géneros y fuentes proceden;
- por qué esos textos son comparables;
- qué limitaciones tiene la muestra.

1.6. Frecuencia absoluta y frecuencia relativa

La frecuencia absoluta es el número de veces que aparece una unidad. Si tokenización aparece 25 veces, su frecuencia absoluta es 25. Esta medida es fácil de entender, pero puede ser engañosa cuando comparamos corpus de distinto tamaño.

La frecuencia relativa corrige ese problema dividiendo la frecuencia por el tamaño del corpus:

frecuencia relativa = frecuencia / total de tokens

Si una palabra aparece 25 veces en 10.000 tokens y 25 veces en 100.000 tokens, no tiene el mismo peso. En el primer corpus ocupa una proporción mucho mayor. Por eso, el código de esta unidad calcula N_{esp} y N_{gen} , es decir, el tamaño del corpus especializado y del corpus general.

La frecuencia relativa no resuelve todos los problemas, pero permite una comparación más justa que la frecuencia absoluta. En extracción terminológica, esta normalización es la puerta de entrada a medidas contrastivas como weirdness.

1.7. Weirdness: rareza relativa y especificidad de dominio

Weirdness es una medida contrastiva. Compara la frecuencia relativa de una unidad en el corpus especializado con su frecuencia relativa en el corpus general. La fórmula didáctica utilizada en la unidad es:

$$\text{weirdness}(t) = (f_{\text{esp}}(t) / N_{\text{esp}}) / ((f_{\text{gen}}(t) / N_{\text{gen}}) + \text{epsilon})$$

El término epsilon es un número muy pequeño que evita la división entre cero. Si una unidad aparece en el corpus especializado y no aparece en el corpus general, el denominador sería cero. Añadir epsilon permite calcular la puntuación sin romper el programa.

La idea de rareza o especificidad se relaciona con enfoques contrastivos de extracción de términos, como el weirdness indexing de Ahmad, Gillam y Tostevin (1999), recogido en revisiones de extracción automática de terminología (Heylen & De Hertog, 2015). También se conecta con la noción de informativeness: una unidad es informativa si ayuda a describir el corpus de dominio mejor que el fondo general.

Weirdness es fácil de explicar y por eso resulta útil en docencia. Pero también tiene límites:

- puede inflar términos que no aparecen en el corpus general;
- depende mucho del tamaño de ambos corpus;
- no mide estabilidad sintáctica;
- no distingue automáticamente términos de nombres propios o expresiones raras;
- puede producir rankings inestables en corpus pequeños.

La puntuación debe verse como una ayuda para ordenar, no como una validación terminológica.

1.8. N-gramas y términos multipalabra

Un n-grama es una secuencia de n tokens consecutivos. Si $n = 1$, hablamos de unigramas. Si $n = 2$, de bigramas. Si $n = 3$, de trigramas. En terminología, los n-gramas son útiles porque muchos términos no son palabras aisladas:

- modelo de lenguaje;
- procesamiento del lenguaje natural;
- traducción automática neuronal;
- etiquetado morfosintáctico;
- frecuencia relativa;
- corpus paralelo.

La extracción de n-gramas es una aproximación simple a los términos multipalabra. No necesita análisis sintáctico, pero puede generar ruido. Si se eliminan stopwords antes de crear n-gramas, modelo de lenguaje puede convertirse en modelo lenguaje. Esa forma puede seguir siendo interpretable, pero no es la forma terminológica natural.

Frantzi et al. (2000), con C-value/NC-value, mostraron la importancia de tratar términos multipalabra y términos anidados. Un término como procesamiento del lenguaje natural contiene unidades menores que también pueden ser candidatas. Los métodos más avanzados intentan ponderar estos casos. En esta unidad no se implementa C-value, pero sí se prepara la intuición: los términos multipalabra son centrales y no basta con contar palabras sueltas.

1.9. Filtrado: frecuencia mínima, stopwords y listas negras

La extracción automática genera mucho ruido. Por eso, después de generar candidatos se aplican filtros. Los filtros de esta unidad son sencillos:

- frecuencia mínima;
- eliminación de stopwords;
- longitud mínima;
- conversión de n-gramas a texto;
- lista negra de unidades demasiado generales.

La frecuencia mínima elimina candidatos que solo aparecen una vez. En corpus grandes esto suele mejorar la precisión, aunque puede eliminar términos raros pero importantes. Las stopwords eliminan palabras funcionales frecuentes. La lista negra permite descartar candidatos que la persona analista considera poco útiles para el dominio, como sistema, texto o análisis si aparecen de forma demasiado general.

Pazienza et al. (2005) subrayan que la extracción terminológica combina enfoques lingüísticos y estadísticos. Esta unidad sigue esa lógica en versión ligera: las estadísticas ordenan candidatos, pero las heurísticas lingüísticas corrigen ruido.

1.10. Métodos transparentes frente a métodos neuronales

La extracción terminológica contemporánea incluye métodos basados en reglas, estadística, aprendizaje automático, modelos neuronales y enfoques híbridos. Tran et al. (2023) revisan avances recientes, y Blandón Andrade et al. (2026) muestran en un mapeo sistemático que los enfoques híbridos son muy frecuentes en la literatura reciente, combinando componentes lingüísticos y estadísticos.

Entonces, ¿por qué esta unidad empieza con un método frecuencial? Porque es interpretable. Si un candidato aparece arriba en el ranking, podemos explicar:

- cuántas veces aparece en el corpus especializado;
- cuántas veces aparece en el corpus general;
- cuál es su frecuencia relativa;
- qué n-grama lo generó;
- qué filtro lo mantuvo;
- qué ejemplos KWIC lo respaldan.

Para aprender PLN desde Lingüística, esa transparencia es fundamental. Antes de usar modelos más complejos, conviene comprender qué significa representar un texto como tokens, contar n-gramas, normalizar por tamaño y contrastar corpus.

1.11. KWIC y validación humana

KWIC significa Key Word In Context. Una concordancia KWIC muestra una unidad con algunas palabras a la izquierda y a la derecha. En extracción terminológica, KWIC ayuda a decidir si un candidato funciona como término en contexto.

Por ejemplo, si el candidato es modelo lenguaje, el KWIC puede revelar que la forma real en el texto era modelo de lenguaje y que la preposición se perdió al eliminar stopwords. Si el candidato es

traducción automática, el KWIC puede mostrar si aparece como concepto técnico estable o como combinación casual.

La validación humana debe revisar al menos:

- si el candidato designa un concepto especializado;
- si la forma está completa;
- si hay variantes;
- si aparece en contextos coherentes;
- si se necesita una definición;
- si hay equivalencias o traducciones relevantes;
- si el término debe entrar en glosario o solo en una lista de palabras clave.

Nazar y Acosta (2023) plantean justamente una producción semiautomática de bases terminológicas: la máquina propone y organiza, pero la terminología final exige decisiones humanas.

1.12. Evaluación y límites de la extracción automática

Evaluar una lista de candidatos terminológicos no es trivial. La precisión pregunta cuántos candidatos recuperados son realmente términos. El recall pregunta cuántos términos reales del dominio ha recuperado el sistema. Pero para calcular recall hace falta saber previamente cuáles son todos los términos correctos del corpus, y eso exige una anotación humana muy costosa. En la práctica docente, la evaluación puede empezar con preguntas cualitativas:

- ¿Los candidatos top parecen propios del dominio?
- ¿Hay expresiones incompletas?
- ¿Se han perdido términos raros?
- ¿El corpus general distorsiona la comparación?
- ¿El filtrado por stopwords elimina partes necesarias?
- ¿Los ejemplos KWIC apoyan la interpretación?

Herramientas recientes como TermoUD integran análisis lingüístico con Universal Dependencies y ranking por C-value, mostrando que la investigación actual busca métodos más robustos y multilingües sin perder la corrección lingüística de los candidatos (Marciniak et al., 2023). La unidad, en cambio, trabaja una versión inicial: más sencilla, pero suficientemente clara para entender la arquitectura del problema.

2. Extracción automática de terminología con Python

2.1. Preparar el entorno

La unidad comienza importando librerías ligeras:

```
import re
from collections import Counter, defaultdict
from math import log
from typing import List, Tuple, Dict
import pandas as pd
```

`re` permite usar expresiones regulares para limpiar texto. `Counter` permite contar tokens y n-gramas. `defaultdict` se importa como estructura útil para acumulaciones, aunque el pipeline principal puede

resolverse con Counter. log queda disponible para métricas que usan logaritmos, aunque la medida central de la unidad es weirdness. List, Tuple y Dict se usan en anotaciones de tipo: ayudan a leer qué recibe y qué devuelve cada función. pandas permite construir tablas de resultados.

Esta combinación muestra una decisión docente: trabajar con herramientas comprensibles y ejecutables en casi cualquier entorno. El objetivo no es depender de una pila pesada de PLN, sino entender el proceso.

2.2. Representar los corpus

El código trabaja con dos listas de textos:

```
corpus_especializado = [  
    "Texto del dominio especializado...",  
    "Otro texto del dominio..."  
]  
  
corpus_general = [  
    "Texto general de contraste...",  
    "Otro texto general..."  
]
```

Cada elemento de la lista representa un documento o fragmento. La separación entre corpus especializado y corpus general es la base del análisis. Si ambos se mezclaran, no podríamos calcular especificidad de dominio.

En un proyecto real, esas listas podrían venir de archivos .txt, columnas de un .csv, resultados de una extracción web o documentos ya segmentados. El formato de lista es didáctico: permite centrarse en el pipeline sin cargar todavía con lectura de archivos.

2.3. Tokenización simple

La función simple_tokenize convierte un texto en una lista de tokens:

```
def simple_tokenize(text: str) -> List[str]:  
    text = text.lower()  
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)  
    text = re.sub(r"^[a-záéíóúüñ\s]", " ", text)  
    text = re.sub(r"\s+", " ", text).strip()  
    tokens = text.split(" ")  
    tokens = [t for t in tokens if len(t) >= 2]  
    return tokens
```

Primero, el texto pasa a minúsculas. Después se eliminan URLs. La expresión regular `r"^[a-záéíóúüñ\s]"` reemplaza por espacios todo lo que no sea una letra española básica o un espacio. Esto elimina puntuación, números y símbolos. Luego se colapsan espacios repetidos y se divide por espacios.

La línea final elimina tokens de un solo carácter. Es una heurística simple para reducir ruido. En otros dominios podría ser problemática: una letra puede ser relevante en química, genética o matemáticas. De nuevo, limpiar es decidir.

2.4. Stopwords y filtrado funcional

La unidad define una lista básica de stopwords en español:

```
STOPWORDS_ES = {
    "de", "la", "que", "el", "en", "y", "a", "los", "del",
    "se", "las", "por", "un", "para", "con", "no", "una"
}

def remove_stopwords(tokens: List[str], stopwords=STOPWORDS_ES) -> List[str]:
    return [t for t in tokens if t not in stopwords]
```

La función recibe una lista de tokens y devuelve otra lista sin palabras funcionales. La comprensión de listas se lee así: conserva cada token *t* si *t* no está en *stopwords*.

En extracción terminológica, este paso suele mejorar los rankings de unigramas, porque evita que palabras como *de*, *la* o *que* dominen los conteos. Pero para términos multipalabra en español puede generar formas incompletas. Por eso la ampliación de la unidad propone conservar stopwords en el interior de los *n*-gramas y filtrar solo los bordes.

2.5. Preprocesar un corpus completo

La función `preprocess_corpus` transforma una lista de textos en una lista plana de tokens:

```
def preprocess_corpus(corpus: List[str]) -> List[str]:
    all_tokens = []
    for doc in corpus:
        toks = simple_tokenize(doc)
        toks = remove_stopwords(toks)
        all_tokens.extend(toks)
    return all_tokens
```

`all_tokens` empieza vacío. El bucle recorre cada documento. Para cada documento, tokeniza, elimina stopwords y añade los tokens a la lista general mediante `.extend()`. La diferencia entre `.append()` y `.extend()` es importante: `.append()` añadiría la lista completa como un elemento; `.extend()` añade sus elementos uno a uno.

Después se aplica a los dos corpus:

```
tokens_esp = preprocess_corpus(corpus_especializado)
tokens_gen = preprocess_corpus(corpus_general)
```

`tokens_esp` y `tokens_gen` son las representaciones limpias sobre las que se calculan frecuencias y *n*-gramas.

2.6. Extraer *n*-gramas

La función `get_ngrams` crea secuencias de *n* tokens consecutivos:

```
def get_ngrams(tokens: List[str], n: int) -> List[Tuple[str, ...]]:
    return [tuple(tokens[i:i+n]) for i in range(len(tokens)-n+1)]
```

Si los tokens son `["modelo", "lenguaje", "grande"]` y *n* = 2, el resultado es:

```
[("modelo", "lenguaje"), ("lenguaje", "grande")]
```

Los n-gramas se guardan como tuplas porque una tupla es inmutable y puede usarse como clave en un Counter. Una lista no podría usarse igual porque es mutable.

Esta función es corta, pero muy potente. Permite generar candidatos de distinta longitud sin escribir tres funciones diferentes.

2.7. Contar n-gramas con Counter

La función `count_ngrams` combina extracción y conteo:

```
def count_ngrams(tokens: List[str], n: int) -> Counter:
    ngrams = get_ngrams(tokens, n)
    return Counter(ngrams)
```

Después se aplica a ambos corpus y a varios tamaños:

```
freq1_esp = count_ngrams(tokens_esp, 1)
freq2_esp = count_ngrams(tokens_esp, 2)
freq3_esp = count_ngrams(tokens_esp, 3)

freq1_gen = count_ngrams(tokens_gen, 1)
freq2_gen = count_ngrams(tokens_gen, 2)
freq3_gen = count_ngrams(tokens_gen, 3)
```

Los nombres son sistemáticos. `freq1_esp` significa frecuencias de unigramas en el corpus especializado. `freq2_gen` significa frecuencias de bigramas en el corpus general. Nombrar bien las variables es parte del trabajo profesional: ayuda a interpretar el código sin perderse.

2.8. Convertir n-gramas a texto

Los n-gramas se cuentan como tuplas, pero para mostrarlos en tablas conviene convertirlos en texto:

```
def ngram_to_text(ng: Tuple[str, ...]) -> str:
    return " ".join(ng)
```

La función une los elementos de la tupla con espacios. Así, `("modelo", "lenguaje")` se convierte en `"modelo lenguaje"`.

Esta conversión es necesaria porque la tabla final debe ser legible para una persona terminóloga. El sistema puede trabajar con tuplas; el informe debe presentar formas comprensibles.

2.9. Filtrar por frecuencia mínima

Antes de rankear, el código filtra candidatos poco frecuentes:

```
def filter_by_min_freq(counter: Counter, min_freq: int) -> Dict[str, int]:
    out = {}
    for ng, f in counter.items():
        if f >= min_freq:
            out[ngram_to_text(ng)] = f
    return out
```

La función recorre un Counter. Si la frecuencia f alcanza el umbral `min_freq`, convierte el n-grama a texto y lo guarda en un diccionario.

Después se aplica por separado:

```
cand1_esp = filter_by_min_freq(freq1_esp, min_freq=2)
cand2_esp = filter_by_min_freq(freq2_esp, min_freq=2)
cand3_esp = filter_by_min_freq(freq3_esp, min_freq=2)
```

El umbral 2 es didáctico. En corpus reales, podría ser 3, 5 o más, según tamaño y objetivo. Subir el umbral aumenta precisión pero puede perder candidatos raros. Bajar el umbral aumenta cobertura pero introduce más ruido.

2.10. Calcular tamaños de corpus y frecuencia relativa

El tamaño de cada corpus se calcula con `len()`:

```
N_esp = len(tokens_esp)
N_gen = len(tokens_gen)
```

Después se define una función para frecuencia relativa:

```
def rel_freq(freq: int, N: int) -> float:
    return freq / N if N > 0 else 0.0
```

La condición evita dividir entre cero. Si N es 0, devuelve 0.0. Esta precaución parece pequeña, pero enseña una práctica importante: el código debe prever datos vacíos o mal preparados.

La frecuencia relativa permite comparar unidades aunque los corpus tengan tamaños distintos. Sin esta normalización, un corpus más grande tendería a producir frecuencias absolutas mayores simplemente por su tamaño.

2.11. Calcular weirdness

La función central del ranking es `compute_weirdness`:

```
def compute_weirdness(
    cand_esp: Dict[str, int],
    freq_gen_counter: Counter,
    N_esp: int,
    N_gen: int,
    epsilon: float = 1e-9
) -> pd.DataFrame:
    rows = []
    gen_text_freq = {ngram_to_text(k): v for k, v in freq_gen_counter.items()}

    for term, f_esp in cand_esp.items():
        f_gen = gen_text_freq.get(term, 0)
        r_esp = rel_freq(f_esp, N_esp)
        r_gen = rel_freq(f_gen, N_gen)
        w = r_esp / (r_gen + epsilon)
        rows.append((term, f_esp, r_esp, f_gen, r_gen, w))

    df = pd.DataFrame(rows, columns=["term", "freq_esp", "rel_esp", "freq_gen",
    "rel_gen", "weirdness"])
    df = df.sort_values("weirdness", ascending=False).reset_index(drop=True)
    return df
```

La función recibe candidatos del corpus especializado y frecuencias del corpus general. Primero convierte las claves del Counter general a texto para poder compararlas con los candidatos. Luego calcula frecuencia absoluta, frecuencia relativa y weirdness para cada término.

La salida es un DataFrame. Sus columnas son:

Columna	Significado
term	Candidato a término
freq_esp	Frecuencia absoluta en corpus especializado
rel_esp	Frecuencia relativa en corpus especializado
freq_gen	Frecuencia absoluta en corpus general
rel_gen	Frecuencia relativa en corpus general
weirdness	Puntuación contrastiva

La tabla se ordena de mayor a menor weirdness. Eso permite revisar primero los candidatos más característicos del corpus especializado según esta medida.

2.12. Aplicar weirdness a unigramas, bigramas y trigramas

La función se aplica tres veces:

```
df_uni = compute_weirdness(cand1_esp, freq1_gen, N_esp, N_gen)
df_bi = compute_weirdness(cand2_esp, freq2_gen, N_esp, N_gen)
df_tri = compute_weirdness(cand3_esp, freq3_gen, N_esp, N_gen)
```

Cada tabla contiene candidatos de una longitud distinta. `df_uni` contiene unigramas, `df_bi` bigramas y `df_tri` trigramas.

Separar primero por longitud ayuda a observar diferencias. Los unigramas suelen ser más frecuentes, pero pueden ser más ambiguos. Los bigramas y trigramas suelen ser más informativos, pero aparecen menos. En terminología, muchas unidades relevantes son multipalabra, así que no conviene quedarse solo con unigramas.

2.13. Unificar candidatos de distinta longitud

Para crear un ranking general se añade una columna `n`:

```
def add_ngram_size(df: pd.DataFrame, n: int) -> pd.DataFrame:
    df = df.copy()
    df["n"] = n
    return df
```

`df.copy()` evita modificar el DataFrame original. La columna `n` indica si el candidato procede de unigramas, bigramas o trigramas. Después se concatenan las tablas:

```
df_all = pd.concat([
    add_ngram_size(df_uni, 1),
    add_ngram_size(df_bi, 2),
    add_ngram_size(df_tri, 3),
], ignore_index=True)

df_all_sorted = df_all.sort_values("weirdness", ascending=False).reset_index(drop=True)
```

`pd.concat` une varias tablas. `ignore_index=True` crea un índice nuevo. La ordenación final produce una lista única de candidatos.

2.14. Refinar con una lista negra

La unidad incorpora una heurística simple:

```
BLACKLIST = {
    "paso", "importante", "sistema", "cantidad",
    "texto", "comparación", "corpus", "uso", "métricas"
}

def contains_blacklisted(term: str, blacklist=BLACKLIST) -> bool:
    toks = term.split()
    return all(t in blacklist for t in toks)
```

La función divide el candidato en palabras y comprueba si todas están en la lista negra. Si todas son demasiado generales, el candidato se elimina:

```
df_clean = df_all_sorted[
    ~df_all_sorted["term"].apply(contains_blacklisted)
].reset_index(drop=True)
```

El símbolo ~ niega la condición. Es decir, conserva las filas para las que `contains_blacklisted` es falso.

Esta heurística no es universal. En un dominio, corpus puede ser un término válido; en otro, puede ser demasiado general. La lista negra debe ajustarse al dominio y justificarse.

2.15. Construir KWIC para candidatos

La función KWIC busca una secuencia exacta dentro de tokens:

```
def kwic(tokens: List[str], term_tokens: List[str], window: int = 5, max_examples: int
= 5) -> List[str]:
    examples = []
    n = len(term_tokens)
    for i in range(len(tokens) - n + 1):
        if tokens[i:i+n] == term_tokens:
            left = " ".join(tokens[max(0, i-window):i])
            hit = " ".join(tokens[i:i+n])
            right = " ".join(tokens[i+n:i+n+window])
            examples.append(f"{left} >>{hit}<< {right}".strip())
            if len(examples) >= max_examples:
                break
    return examples
```

`term_tokens` es la forma del candidato separada en palabras. Si el candidato es modelo lenguaje, `term_tokens` será ["modelo", "lenguaje"]. La función recorre todas las posiciones posibles y compara segmentos de longitud `n`.

Cuando encuentra una coincidencia, toma contexto izquierdo y derecho. El término aparece marcado entre `>>` y `<<`. Este formato permite localizar visualmente el candidato dentro de su entorno.

El KWIC ayuda a pasar del ranking automático a la validación terminológica. Un candidato con score alto pero contextos extraños debe revisarse con cautela.

2.16. Crear un miniglosario automático

La función `build_mini_glossary` transforma el ranking en una tabla más útil:

```
def build_mini_glossary(df_ranked: pd.DataFrame, tokens_esp: List[str], top_n: int =
15) -> pd.DataFrame:
    rows = []
    for _, row in df_ranked.head(top_n).iterrows():
        term = row["term"]
        term_tokens = term.split()
        examples = kwic(tokens_esp, term_tokens, window=5, max_examples=3)
        rows.append({
            "term": term,
            "n": row["n"],
            "weirdness": row["weirdness"],
            "freq_esp": row["freq_esp"],
            "freq_gen": row["freq_gen"],
            "kwic_examples": examples
        })
    return pd.DataFrame(rows)
```

La función recorre los primeros candidatos del ranking. Para cada candidato, recupera su tamaño, puntuación, frecuencias y ejemplos KWIC. El resultado es una tabla que se parece más a una primera ficha de glosario.

No es una base terminológica final. Le faltan definiciones, equivalencias, variantes, notas y validación. Pero sí ofrece una estructura inicial muy útil para una terminología.

2.17. Imprimir el glosario de forma legible

La función `print_glossary` prepara una salida más fácil de leer:

```
def print_glossary(glossary_df: pd.DataFrame):
    for i, row in glossary_df.iterrows():
        print(f"#{i+1} {row['term']} (n={row['n']})")
        print(f"weirdness={row['weirdness']:.2f} | esp={row['freq_esp']} | "
              f"gen={row['freq_gen']}")
        for ex in row["kwic_examples"]:
            print("-", ex)
```

`iterrows()` recorre las filas del `DataFrame`. Las cadenas con `f"..."` permiten insertar valores de variables dentro del texto. La expresión `{row['weirdness']:.2f}` muestra la puntuación con dos decimales.

La presentación importa. Una lista terminológica no solo debe calcularse; debe leerse. Si el equipo humano no entiende la salida, no podrá validarla.

2.18. Conservar stopwords dentro de términos

La ampliación opcional introduce una mejora metodológica: no eliminar stopwords antes de crear n-gramas, pero sí filtrar candidatos que empiezan o terminan con stopword.

La razón es lingüística. En español, muchos términos multipalabra incluyen conectores internos:

- modelo de lenguaje;
- procesamiento del lenguaje natural;
- extracción de términos;
- lingüística de corpus;
- análisis de colocaciones.

Si eliminamos `de`, `del` o `la` antes de formar n-gramas, el candidato queda truncado. La ampliación propone:

```
def ngram_good_edges(ng_text: str, stopwords=STOPWORDS_ES) -> bool:
    toks = ng_text.split()
    return (toks[0] not in stopwords) and (toks[-1] not in stopwords)
```

La función acepta candidatos que no empiezan ni terminan con stopword, aunque puedan contener stopwords en el interior. Es una heurística simple y bastante realista para español.

Esta mejora muestra una lección central: el preprocesamiento no es una receta fija. Debe adaptarse al objetivo lingüístico. Para unigramas, eliminar stopwords puede ser razonable. Para términos multipalabra, conservar algunas puede ser necesario.

2.19. Cómo leer los resultados sin adelantar la práctica

Antes de aplicar el código en clase, el alumnado debe saber cómo leer sus salidas:

- Un ranking alto indica prioridad de revisión, no verdad terminológica.
- Una frecuencia alta puede deberse a repetición temática, no necesariamente a valor terminológico.
- Un candidato multipalabra puede estar incompleto por la limpieza.
- Un término raro puede quedar fuera si el umbral de frecuencia es alto.
- Un corpus general pequeño puede inflar weirdness.
- El KWIC debe revisarse antes de aceptar una entrada de glosario.

El informe profesional debe formular resultados con prudencia. En vez de escribir «estos son los términos del dominio», es mejor escribir «estos son candidatos priorizados por frecuencia relativa y contraste con corpus general». Esa frase es más larga, pero mucho más correcta.

Errores frecuentes de aprendizaje

- Confundir término con palabra frecuente.
- Pensar que weirdness valida automáticamente una entrada terminológica.
- Comparar frecuencias absolutas entre corpus de tamaños distintos.
- Olvidar que un corpus general pequeño puede inflar puntuaciones.
- Eliminar stopwords antes de formar n-gramas sin revisar si se deforman términos en español.
- Creer que todos los términos relevantes aparecen muchas veces.
- No distinguir unigramas, bigramas y trigramas al interpretar rankings.
- Aceptar candidatos sin mirar ejemplos KWIC.
- Usar una lista negra sin justificarla.
- Presentar un miniglosario automático como glosario final validado.

Síntesis final

Esta unidad introduce la extracción automática de terminología como una tarea de apoyo al trabajo terminológico humano. El objetivo no es que Python decida por sí solo qué unidades son términos, sino que genere candidatos razonables a partir de datos observables.

El marco teórico ha mostrado que la terminología combina lengua, conocimiento especializado y comunicación profesional. También ha situado la extracción automática en una tradición de investigación que distingue termhood, unithood, frecuencia, especificidad, n-gramas, C-value/NC-value, weirdness y validación humana.

La parte de Python enseña un pipeline transparente: preparar corpus, tokenizar, filtrar stopwords, extraer n-gramas, contar frecuencias, normalizar por tamaño, calcular weirdness, unificar rankings, depurar candidatos y generar ejemplos KWIC. Cada bloque produce una representación distinta del corpus y exige decisiones lingüísticas.

El aprendizaje más importante es metodológico: una lista automática de candidatos no es un glosario. Es una herramienta de priorización. Su valor depende de la calidad de los corpus, de los filtros, de las métricas y de la revisión experta.

Cierre narrativo

Al terminar la simulación, LexiData Solutions no entrega al cliente una base terminológica cerrada. Entrega un primer detector de candidatos: transparente, explicable y revisable. El equipo puede enseñar por qué un candidato aparece arriba en el ranking, cuántas veces aparece en el corpus especializado, si aparece en el corpus general y qué contextos KWIC lo respaldan.

La terminóloga humana sigue tomando la decisión final. Pero ahora no empieza desde una página en blanco. Tiene una lista priorizada, ejemplos de uso y una justificación cuantitativa inicial. Esa es la aportación real del PLN en este escenario: no sustituir la lectura experta, sino hacerla más eficiente, más trazable y más documentada.

En el aula, los casos prácticos permitirán modificar umbrales, sustituir corpus y discutir qué candidatos merecen entrar en un glosario. Este manual te prepara para leer el código antes de hacerlo: entender qué mide cada función, qué presupone cada filtro y qué tipo de afirmación puede sostener cada salida.

Referencias

- Ahmad, K., Gillam, L. y Tostevin, L. (1999). Weirdness indexing for logical document extrapolation and retrieval (WILDER). In E. M. Voorhees & D. K. Harman (Eds.), *The Eighth Text Retrieval Conference (TREC-8)* (pp. 717-724). National Institute of Standards and Technology.
- Blandón Andrade, J. C., Medina Otálvaro, C. M., Zapata Jaramillo, C. M. y Morales Ríos, A. (2026). Approaches, tools, algorithms, and methods for automatic term extraction: A systematic literature mapping. *Journal of Intelligent & Fuzzy Systems*, 50(6), 1803-1823. <https://doi.org/10.1177/18758967251392652>
- Cabré, M. T. (1999). *Terminology: Theory, methods and applications*. John Benjamins. <https://doi.org/10.1075/tlrp.1>
- Cabré, M. T. (2003). Theories of terminology: Their description, prescription and explanation. *Terminology*, 9(2), 163-199. <https://doi.org/10.1075/term.9.2.03cab>
- Frantzi, K. T., Ananiadou, S. y Mima, H. (2000). Automatic recognition of multi-word terms: The C-value/NC-value method. *International Journal on Digital Libraries*, 3(2), 115-130. <https://doi.org/10.1007/s007999900023>
- Heylen, K. y De Hertog, D. (2015). Automatic term extraction. En H. J. Kockaert y F. Steurs (Eds.), *Handbook of terminology* (Vol. 1). John Benjamins. <https://doi.org/10.1075/hot.1.aut1>
- Kageura, K. y Umino, B. (1996). Methods of automatic term recognition: A review. *Terminology*, 3(2), 259-289. <https://doi.org/10.1075/term.3.2.03kag>
- Marciniak, M., Rychlik, P. y Mykowiecka, A. (2023). TermoUD: A language-independent terminology extraction tool. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (pp. 178-186). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.eacl-demo.21>
- Nazar, R. y Acosta, N. (2023). Termout: A tool for the semi-automatic creation of term databases. In *Proceedings of the Workshop on Computational Terminology in NLP and Translation Studies (ConTeNTS) Incorporating the 16th Workshop on Building and Using Comparable Corpora (BUCC)* (pp. 9-18). INCOMA Ltd. <https://aclanthology.org/2023.contents-1.2/>
- Pazienza, M. T., Pennacchiotti, M. y Zanzotto, F. M. (2005). Terminology extraction: An analysis of linguistic and statistical approaches. In S. Sirmakessis (Ed.), *Knowledge mining* (pp. 255-279). Springer. https://doi.org/10.1007/3-540-32394-5_20
- Tran, H. T. H., Martinc, M., Caporusso, J., Doucet, A. y Pollak, S. (2023). *The recent advances in automatic term extraction: A survey*. arXiv. <https://arxiv.org/abs/2301.06767>
- Verberne, S., Sappelli, M., Hiemstra, D. y Kraaij, W. (2016). Evaluation and analysis of term scoring methods for term extraction. *Information Retrieval Journal*, 19, 510-545. <https://doi.org/10.1007/s10791-016-9286-2>